Propuesta Técnica: Gestión de Datos para IA - NYC Taxi Trips (2018)

Nombres:

Vicente Cossio

Fernando Guzmán

Asignatura: Gestión de Datos para IA

Sección: 800D

Docente: Felipe Zambrano

Fecha: 10/04/2026

Caso de Estudio: Google Cloud NYC Taxi Trips

1. Arquitectura Escalable: Pipeline Híbrido de IA

Para el procesamiento del dataset histórico de NYC Taxi Trips del año 2018, y considerando el enfoque de un proyecto semestral que integra el ciclo de vida de modelos de Inteligencia Artificial, se ha seleccionado la Arquitectura Pipeline Híbrido de IA. Esta arquitectura es idónea para gestionar datos históricos masivos que alimentarán procesos de Machine Learning, combinando la robustez del procesamiento por lotes con la flexibilidad necesaria para la experimentación y despliegue de modelos de IA.

Justificación Técnica

La Arquitectura Pipeline Híbrido de IA se justifica por su capacidad para integrar de manera eficiente el análisis exploratorio de datos (EDA), la ingeniería de características, el entrenamiento de modelos y su posterior despliegue, todo ello sobre un volumen considerable de datos históricos. Este enfoque permite:

Optimización para MLOps: Facilita la creación de un flujo de trabajo estructurado para el desarrollo, entrenamiento, evaluación y despliegue de modelos de IA, lo cual es crucial en un proyecto semestral enfocado en la gestión de datos para IA.

Flexibilidad en el Procesamiento: Aunque los datos son históricos, la arquitectura permite la experimentación con diferentes algoritmos y transformaciones, utilizando herramientas como BigQuery para la preparación inicial y Jupyter Notebooks (con Spark/Pandas) para el procesamiento intensivo y el entrenamiento de modelos.

Escalabilidad para Datos Masivos: Aprovecha la capacidad de BigQuery para manejar petabytes de datos y la escalabilidad de Spark para el procesamiento distribuido, asegurando que el volumen de datos de un año completo no sea una limitación.

Integración Nativa con Google Cloud: Se alinea perfectamente con el uso de Google Cloud BigQuery y Google Colab (Jupyter Notebooks), proporcionando un ecosistema cohesivo para el desarrollo del proyecto.

Descarte de otras opciones

Arquitectura Lakehouse: Aunque es una excelente opción para la gestión de datos masivos

y ofrece capacidades ACID sobre Data Lakes, su enfoque principal es la unificación de Data Warehouses y Data Lakes para análisis de negocio y gobernanza de datos. En el contexto de un proyecto semestral con un énfasis explícito en el ciclo de vida de la IA, el Lakehouse podría introducir una capa de complejidad adicional en la gestión de metadatos y transacciones que no es el foco principal del desarrollo de modelos de IA. El Pipeline Híbrido de IA, en cambio, se centra más directamente en el flujo de datos hacia y desde los modelos de IA.

Arquitectura Lambda: Se descarta categóricamente debido a que su diseño está orientado a manejar dos flujos en paralelo: una capa de velocidad (streaming) y una capa de lote (batch). Dado que el dataset es puramente histórico y estático (datos de 2018), no existe una necesidad de procesamiento en tiempo real. La implementación de una capa de velocidad en este escenario sería redundante, generaría una complejidad técnica innecesaria y un costo de infraestructura injustificado, desviándose del objetivo de procesar datos históricos para IA.

2. Interoperabilidad y Seguridad

Interoperabilidad

Para asegurar que los datos sean consumibles por cualquier modelo de IA futuro y facilitar el intercambio entre las diferentes etapas del pipeline, se utilizarán los siguientes formatos:

Parquet: Este formato de almacenamiento columnar es altamente eficiente para la lectura masiva de datos estructurados, lo que lo hace ideal para el entrenamiento de modelos de IA. Es compatible de forma nativa con herramientas como Spark, Pandas y BigQuery, asegurando una interoperabilidad fluida a lo largo del pipeline.

Seguridad (Datos en Reposo)

Considerando que los datos residen en Google Cloud Storage y BigQuery, y que son datos históricos en reposo, se aplicarán las siguientes medidas para garantizar su seguridad:

Encriptación en Reposo: Todos los datos almacenados en Google Cloud Storage y BigQuery se encriptarán automáticamente mediante claves gestionadas por Google. Adicionalmente, se puede optar por Claves de Encriptación Administradas por el Cliente (CMEK) para un control más granular sobre las claves de encriptación.

Control de Acceso Basado en Roles (IAM): Se implementará el principio de "mínimo privilegio", asignando roles y permisos específicos a los usuarios y servicios que interactúan con los datos. Esto asegura que solo las entidades autorizadas puedan acceder, modificar o eliminar los datos.

Enmascaramiento y Tokenización de Datos: Si el dataset contuviera información sensible (aunque el dataset de taxis de NYC es público, en un escenario real podría haberla), se aplicarían técnicas de enmascaramiento o tokenización para proteger los datos antes de que sean utilizados por los modelos de IA, especialmente en entornos de desarrollo o pruebas.

Auditoría y Monitoreo: Se habilitarán los registros de auditoría de Google Cloud (Cloud Audit Logs) para monitorear el acceso y las operaciones realizadas sobre los datos, permitiendo detectar y responder a actividades sospechosas.

3. Entorno Técnico y CI/CD

Portabilidad con Docker

Se utilizará Docker para contenerizar el entorno de desarrollo y ejecución, garantizando la portabilidad y reproducibilidad del pipeline de IA. La imagen Docker incluirá:

Sistema Operativo Base: Una distribución ligera de Linux.

Runtime de Python: Versión 3.11, con todas las librerías necesarias preinstaladas (`pandas` , `numpy` , `scikit-learn` , `pyspark` , `google-cloud bigquery` , `google-cloud-storage` , etc.).

Jupyter Lab: Para proporcionar un entorno interactivo consistente para el desarrollo y la experimentación con los Notebooks.

Configuración de Credenciales: Mecanismos seguros para la inyección de credenciales de Google Cloud, evitando su hardcoding en la imagen.

Esta estrategia asegura que cualquier desarrollador pueda replicar el entorno exacto de trabajo, eliminando problemas de compatibilidad y facilitando la colaboración.

Integración de Herramientas en el Flujo

1. **Google Cloud BigQuery (EDA y Pre-procesamiento)**: Se utilizará BigQuery para el

Análisis Exploratorio de Datos (EDA) inicial mediante SQL, permitiendo identificar patrones, anomalías y la calidad de los datos a gran escala. También servirá para realizar un pre-procesamiento inicial y filtrado de los datos de 2018 antes de su extracción para tareas más complejas.

2. **Jupyter Notebooks (Google Colab con Python/Pandas/Spark)**: Los Notebooks serán el corazón del pipeline para la transformación de datos, ingeniería de características y el entrenamiento de modelos de IA. Se conectarán a BigQuery para extraer los datos pre-procesados y utilizarán librerías como Pandas para manipulaciones en memoria o PySpark para procesamiento distribuido de volúmenes mayores, aprovechando la capacidad de Google Colab para escalar recursos computacionales.

Estructura de README.md

El archivo `README.md` será el punto de entrada principal para cualquier colaborador o revisor del proyecto, detallando:

Descripción del Proyecto: Contexto del dataset de NYC Taxi Trips de 2018 y los objetivos del proyecto de IA.

Arquitectura Seleccionada: Una descripción concisa de la Arquitectura Pipeline Híbrido de IA y su justificación para este caso.

Requisitos del Sistema: Lista de dependencias (Docker, Git, Cuenta de Google Cloud Platform con permisos específicos).

Instrucciones de Instalación y Configuración: Pasos detallados para clonar el repositorio, construir la imagen Docker, configurar las credenciales de GCP y levantar el entorno de Jupyter Lab (`docker-compose up`).

Estructura del Repositorio: Explicación de la organización de directorios (`/data` , `/notebooks` , `/scripts` , `/docker` , `/models`), indicando el propósito de cada uno.

Guía de Uso: Cómo ejecutar los Notebooks, scripts de transformación y entrenar los modelos.

Pipeline CI/CD (GitHub Actions)

Se configurará un flujo de Integración Continua/Despliegue Continuo (CI/CD) utilizando GitHub Actions para automatizar y estandarizar el ciclo de desarrollo del pipeline de IA:

1. **Validación de Código (Linting y Tests)**: Ante cada push o pull request, se ejecutarán

linters (ej. `flake8` , `pylint`) para Python y tests unitarios/de integración para los scripts de transformación y módulos de modelos. También se validarán las queries SQL de BigQuery.

2. **Construcción de Imagen Docker**: Si las validaciones son exitosas, se construirá automáticamente una nueva imagen Docker con las últimas dependencias y código, y se etiquetará con la versión correspondiente.

3. **Despliegue en Entorno de Desarrollo/Staging**: La imagen Docker construida se desplegará en un entorno de desarrollo o staging en Google Cloud, donde se ejecutarán pruebas de integración y se validará el funcionamiento del pipeline completo con datos de prueba. Esto podría incluir la ejecución de los Notebooks en modo no interactivo o scripts de entrenamiento.

4. **Notificaciones**: Se enviarán notificaciones (ej. a Slack o correo electrónico) sobre el estado de las ejecuciones del pipeline CI/CD.

4. Plan de Gestión (Enfoque PMBOK Predictivo)

El proyecto se gestionará bajo un enfoque PMBOK. Esto permite una planificación detallada y una ejecución secuencial de las fases.

Estructura de Desglose del Trabajo (WBS)

Fase	Entregables Clave
1. Planificación	Acta de Constitución del Proyecto, Plan de Gestión de Riesgos, Cronograma Detallado del Proyecto, Plan de Comunicación.
2. Diseño Técnico	Documento de Diseño de Arquitectura (Pipeline Híbrido de IA), Diseño Detallado del DFD Batch, Diccionario de Datos Completo, Especificaciones de Interoperabilidad y Seguridad.
3. Desarrollo Técnico	Scripts de Extracción y Carga (BigQuery), Notebooks de EDA y Feature Engineering, Módulos de Entrenamiento de Modelos de IA, Configuración de Entorno Docker, Scripts de CI/CD (GitHub Actions).

4. Documentación y Cierre	Informe Técnico Final, Manual de Usuario/Operaciones del Pipeline, Repositorio de Código Documentado (README.md), Acta de Cierre del Proyecto.

Hitos Críticos

Hito 1: Aprobación del Diseño de Arquitectura y Diccionario de Datos (Fin de Fase 2). Este hito marca la finalización de la fase de diseño y la aprobación de la estrategia técnica, permitiendo el inicio del desarrollo. Es crucial para asegurar que todos los stakeholders estén alineados con la visión técnica del proyecto.

Hito 2: Pipeline de Datos Funcional y Modelo Base Entrenado (Fin de Fase 3 - Desarrollo Técnico). Este hito valida que el pipeline de datos puede procesar los datos históricos de 2018 hasta la etapa de entrenamiento, y que un modelo de IA inicial ha sido entrenado con éxito. Representa un punto de no retorno significativo en el desarrollo técnico.

Riesgo Técnico Importante y Estrategia de Mitigación

Riesgo: Degradación del rendimiento de los Notebooks de Jupyter/Colab al procesar grandes volúmenes de datos históricos. Aunque BigQuery es escalable, la extracción y procesamiento intensivo en Notebooks (especialmente con Pandas en memoria) puede llevar a cuellos de botella o fallos por falta de recursos, afectando la eficiencia del pipeline de IA.

Estrategia de Mitigación:

1. **Optimización de Extracción**: Utilizar consultas SQL optimizadas en BigQuery para reducir el volumen de datos extraídos a los Notebooks, realizando agregaciones o filtrados previos.
2. **Uso Estratégico de PySpark**: Para transformaciones que no puedan ser manejadas eficientemente por Pandas, migrar a PySpark dentro de los Notebooks, aprovechando la capacidad de procesamiento distribuido de Spark en Google Colab (si se usa la versión Pro o Enterprise) o en un clúster de Dataproc.
3. **Muestreo Inteligente**: Implementar técnicas de muestreo estratificado o aleatorio para el desarrollo y pruebas iniciales de los modelos, trabajando con subconjuntos representativos del dataset completo antes de escalar al volumen total.
4. **Monitoreo de Recursos**: Implementar monitoreo de uso de CPU, memoria y disco en

los entornos de Notebooks para identificar y diagnosticar cuellos de botella de rendimiento de manera proactiva.

5. DFD y Diccionario de Datos

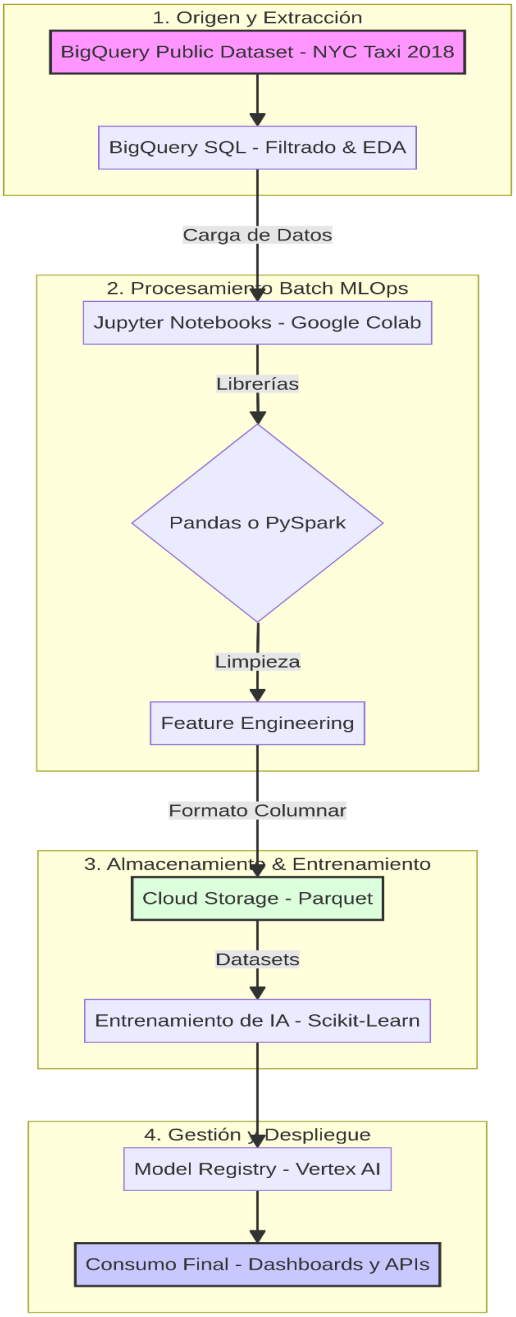
Diagrama de Flujo de Datos (DFD) - Proceso Batch

El Diagrama de Flujo de Datos (DFD) describe el movimiento y la transformación de los datos a través del pipeline, desde su origen hasta el consumo por los modelos de IA.
Este proceso es fundamentalmente Batch, operando sobre el dataset histórico completo de 2018.

1. **Origen de Datos (BigQuery NYC Taxi 2018)**: Los datos brutos de los viajes de taxi de NYC para el año 2018 se encuentran almacenados en el dataset público de Google Cloud BigQuery. Este es el punto de partida para la extracción.
2. **Extracción y Pre-procesamiento (BigQuery SQL)**: Mediante consultas SQL en BigQuery, se realiza una extracción inicial de los datos relevantes. En esta etapa, se pueden aplicar filtros básicos, selección de columnas y algunas transformaciones de limpieza preliminares para reducir el volumen y mejorar la calidad de los datos antes de pasarlos a los Notebooks.
3. **Transformación y Feature Engineering (Jupyter Notebooks / Spark/Pandas)**: Los datos extraídos se cargan en Jupyter Notebooks (ejecutándose en Google Colab). Aquí, se llevan a cabo transformaciones más complejas, limpieza de valores atípicos, imputación de nulos, creación de nuevas características (Feature Engineering) y normalización. Se utilizan librerías como Pandas para operaciones en memoria o PySpark para procesamiento distribuido, dependiendo del volumen y la complejidad.
4. **Almacenamiento Intermedio (Google Cloud Storage / Parquet/Avro)**: Los datos transformados y las características ingenierizadas se almacenan en Google Cloud Storage en formatos optimizados como Parquet o Avro. Este almacenamiento actúa como una capa persistente y optimizada para el consumo por los modelos de IA, permitiendo la reusabilidad y el versionamiento de los datasets de entrenamiento.
5. **Entrenamiento de Modelos de IA (Jupyter Notebooks / Frameworks ML)**: Los modelos de Machine Learning (ej. scikit-learn, TensorFlow, PyTorch) consumen los datasets preparados desde Google Cloud Storage. En esta etapa, se realiza el entrenamiento, la validación y la evaluación de los modelos de IA dentro de los Jupyter Notebooks.
6. **Model Registry (Google Cloud AI Platform Unified)**: Los modelos entrenados y validados

se registran en un Model Registry (como Google Cloud AI Platform Unified) para su gestión, versionamiento y despliegue.

7. **Despliegue y Consumo (APIs / Dashboards)**: Los modelos registrados pueden ser desplegados como APIs (ej. Google Cloud Endpoints) para inferencia en tiempo real o por lotes, o sus predicciones pueden ser consumidas por dashboards de visualización (ej. Looker Studio, Tableau) para análisis de negocio o monitoreo.



Diccionario de Datos (Campos Fundamentales)

La siguiente tabla detalla al menos 10 campos fundamentales del dataset de viajes en taxi de NYC, esenciales para el análisis y el entrenamiento de modelos de IA, incluyendo su tipo de dato, restricciones y una descripción clara:

Campo	Tipo de Dato	Restricciones	Descripción
vendor_id	STRING	No nulo, valores discretos (ej. 'CMT', 'VTS')	Identificador del proveedor de servicios de taxi.
pickup_datetime	DATETIME	Formato ISO 8601, no nulo	Fecha y hora exacta en que el pasajero fue recogido.
dropoff_datetime	DATETIME	Formato ISO 8601, > pickup_datetime , no nulo	Fecha y hora exacta en que el pasajero fue dejado.
passenger_count	INTEGER	>= 0, <= 6 (típicamente)	Número de pasajeros en el viaje, reportado por el conductor.
trip_distance	FLOAT	>= 0.0	Distancia total del viaje en millas.
pickup_longitude	FLOAT	Rango geográfico de NYC	Longitud geográfica del punto de recogida.
pickup_latitude	FLOAT	Rango geográfico de NYC	Latitud geográfica del punto de recogida.
dropoff_longitude	FLOAT	Rango geográfico de NYC	Longitud geográfica del punto de destino.
dropoff_latitude	FLOAT	Rango geográfico de NYC	Latitud geográfica del punto de destino.

fare_amount	FLOAT	>= 0.0	Monto base de la tarifa del viaje, medido por el taxímetro.
tip_amount	FLOAT	>= 0.0	Monto de la propina pagada por el pasajero.
total_amount	FLOAT	>= fare_amount	Monto total cobrado al pasajero, incluyendo tarifa, extras y propina.

Enlace al repositorio oficial del proyecto aquí: [Repositorio NYC Taxi Trips]